



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE VERACRUZ
INGENIERÍA EN SISTEMAS COMPUTACIONALES

Manual de Usuario

Ofelia Gutiérrez Giraldi

6J1A - Lenguajes y Autómatas I

Equipo 6:

Quintana Pacheco Ángel - 23020808

Sánchez Herrera César Antonio - 23021018

Vargas Valle Marco Antonio - 23020753

ÍNDICE

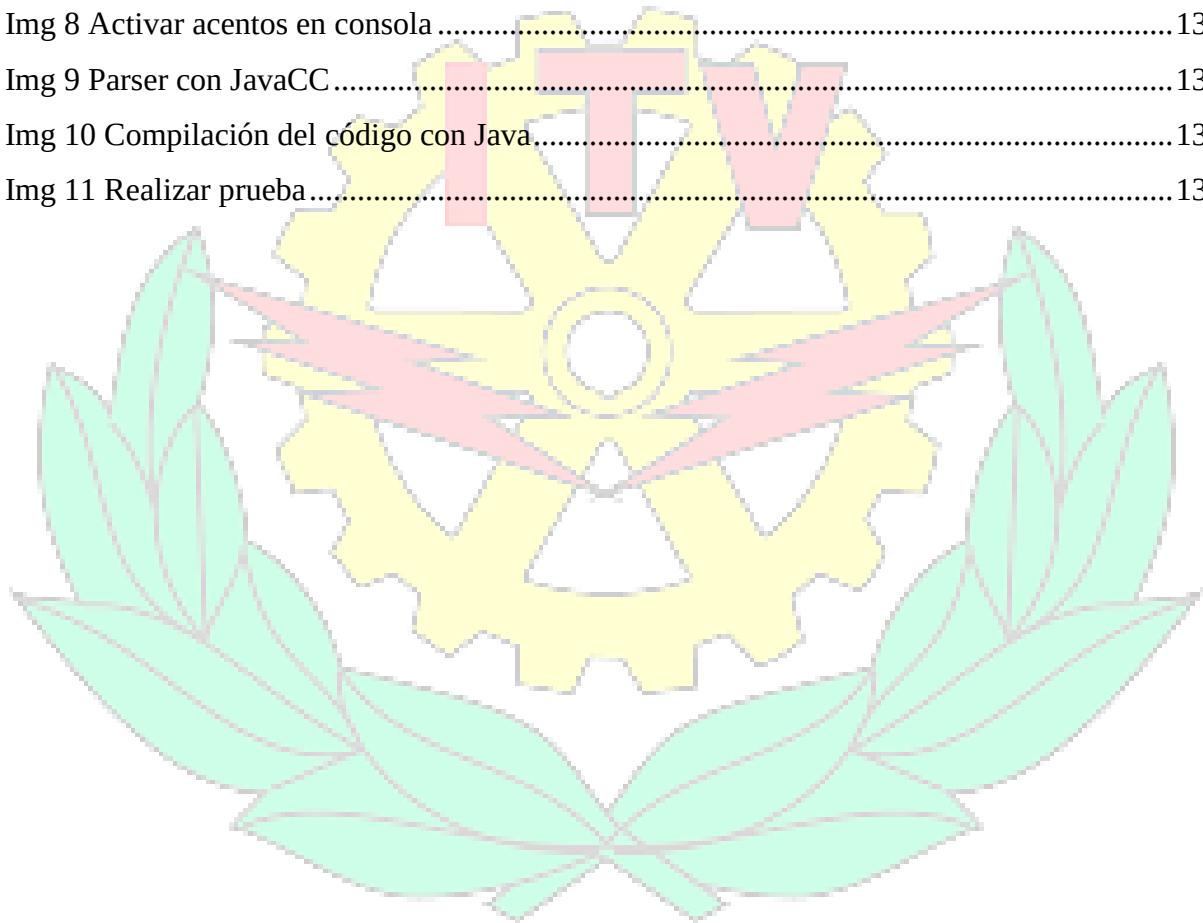
INTRODUCCIÓN	1
REQUERIMIENTOS DEL SISTEMA.....	3
Requerimientos mínimos.....	3
Requerimientos recomendados	4
INSTALACIÓN Y CONFIGURACIÓN DE JAVACC	5
1. Descarga	5
2. Instalación	5
3. Configuración de Variables de Entorno	6
INSTALACIÓN Y CONFIGURACIÓN DE JDK.....	7
1. Descarga	7
2. Instalación	7
3. Configuración de Variables de Entorno	7
CONFIGURACIÓN DEL ENTORNO DE TRABAJO	9
COMPILACIÓN Y EJECUCIÓN PASO A PASO	12
SENTENCIAS DEL LENGUAJE	14
ESTRUCTURA DE PROGRAMA.....	14
COMENTARIOS	14
TIPOS DE DATOS	15
OPERADORES.....	16
DELIMITADORES	17
DECLARACIÓN DE VARIABLES	17
DECLARACIÓN DE CONSTANTES	17
ASIGNACIÓN:	18
INCREMENTO Y DECREMENTO	18
ARREGLOS Y MATRICES:.....	18
IMPRESIÓN A PANTALLA	19

LECTURA DE DATOS.....	19
CONDICIONAL SI / CONTRARIO / SINO.....	20
BUCLE PARA	21
BUCLE MIENTRAS	21
ERRORES LÉXICOS	22
ERRORES SINTÁCTICOS.....	24
ERRORES DE INSTALACIÓN Y CONFIGURACIÓN	28
ERRORES DE EJECUCIÓN	30
PREVENCIÓN DE ERRORES.....	31
REFERENCIAS BIBLIOGRÁFICAS.....	32



ÍNDICE DE IMÁGENES

Img 1 Ubicación carpeta OS(C:):.....	9
Img 2 Ubicación carpeta “Usuarios”.	9
Img 3 Identificar carpeta con nombre de usuario.	10
Img 4 Creación de la carpeta.	10
Img 5 Archivo jj en la carpeta “EasyCompiler”.	11
Img 6 Entrar a consola	12
Img 7 Entrar a carpeta del compilador.....	12
Img 8 Activar acentos en consola	13
Img 9 Parser con JavaCC	13
Img 10 Compilación del código con Java.....	13
Img 11 Realizar prueba.....	13



INTRODUCCIÓN

Bienvenido al manual de usuario de EasyCompiler. Este documento ha sido elaborado para acompañarlo en el uso de la herramienta, partiendo de los conceptos fundamentales hasta alcanzar un manejo avanzado de sus funciones. Tanto si se inicia en el ámbito de la programación como si cuenta con experiencia previa como desarrollador, en las siguientes páginas encontrará la información necesaria para aprovechar al máximo las capacidades del compilador.

EasyCompiler es una herramienta didáctica concebida para traducir instrucciones escritas en el lenguaje de programación EasyScript a código ejecutable. Su desarrollo responde a la necesidad de establecer un puente efectivo entre el código fuente redactado por el programador y el lenguaje que la máquina puede procesar.

El proyecto surge con un enfoque específico en EasyScript, asumiendo la tarea de convertir instrucciones legibles para las personas en órdenes que el ordenador pueda ejecutar de manera eficiente. La motivación principal que impulsa su creación es simplificar el proceso de compilación, haciéndolo accesible, comprensible y con un marcado carácter formativo.

Este manual, así como la herramienta que documenta, está dirigido a los siguientes perfiles:

- Estudiantes y aficionados que deseen aprender un lenguaje de programación fácil y en su idioma.
- Personas interesadas en explorar nuevos lenguajes de programación, particularmente aquellos que buscan familiarizarse con la sintaxis y estructura de EasyScript.
- Desarrolladores con curiosidad por experimentar con un proceso de compilación sencillo y adecuadamente documentado.
- Educadores que requieran una herramienta clara para impartir conceptos relacionados con el análisis léxico, el análisis sintáctico y la generación de código.

No es preciso ser un especialista en compiladores para utilizar EasyCompiler; únicamente se requiere interés y disposición para el aprendizaje.

EasyCompiler presenta un conjunto de cualidades que lo distinguen y facilitan su manejo:

- Enfoque exclusivo en EasyScript, al haber sido diseñado específicamente para traducir este lenguaje, lo que asegura una integración coherente y un rendimiento ajustado a sus particularidades.
- Carácter didáctico y transparente, al permitir el seguimiento del proceso de compilación de forma clara, mostrando las distintas etapas de análisis y transformación del código.
- Proceso simplificado, al reducir la complejidad habitual de otros compiladores y ofrecer una experiencia fluida desde el código fuente hasta la obtención del resultado ejecutable.
- Portabilidad, al tratarse de un compilador independiente que puede utilizarse en diferentes entornos sin requerir configuraciones complejas.
- Facilidad de uso, con una interfaz pensada para que la interacción resulte intuitiva y directa.
- Mensajes de error claros, que asisten al programador en la identificación y corrección de errores en el código EasyScript mediante una retroalimentación descriptiva y útil.

REQUERIMIENTOS DEL SISTEMA

Para garantizar el correcto funcionamiento de EasyCompiler, es necesario que el equipo donde se ejecute cumpla con los siguientes requisitos:

Requerimientos mínimos

Los requerimientos mínimos permiten ejecutar el compilador de manera funcional, aunque el rendimiento puede verse afectado al compilar programas más complejos o de mayor tamaño.

Componente	Especificación
Procesador	Intel Core i3 o equivalente (1.5 GHz)
Memoria RAM	2 GB
Almacenamiento	50 MB de espacio libre en disco
Sistema Operativo	Windows 7/8/10/11
Java	Java Development Kit (JDK) 8 o superior
JavaCC	JavaCC 7.0.13
Terminal	Acceso a línea de comandos (CMD)
Editor de texto	Bloc de notas
Resolución de pantalla	1024 x 768 píxeles

Requerimientos recomendados

Los requerimientos recomendados aseguran una experiencia óptima, especialmente al trabajar con programas extensos o al utilizar el compilador de forma intensiva en entornos educativos.

Componente	Especificación
Procesador	Intel Core i5 o superior (2.0 GHz o más)
Memoria RAM	4 GB o más
Almacenamiento	100 MB de espacio libre en disco
Sistema Operativo	Windows 10/11
Java	Java Development Kit (JDK) 17 o superior
Javacc	JavaCC 7.0.13
Terminal	cmd.exe
Editor de texto	Notepad++
Resolución de pantalla	1920 x 1080 píxeles o superior

INSTALACIÓN Y CONFIGURACIÓN DE JAVACC

1. Descarga

Paso 1: Accede al sitio oficial de JavaCC: <https://javacc.github.io/javacc/>

Paso 2: Descarga el código fuente en formato ZIP haciendo clic en la opción JavaCC 7.0.13 ZIP (ubicada en el centro de la página).

Paso 3: En esa misma página principal, haz clic en el botón "View on GitHub".

Paso 4: Dentro del repositorio de GitHub, desplázate hacia abajo hasta localizar el archivo README.

Paso 5: En el README, busca el apartado "Version 7" y haz clic en el enlace "Binaries" para descargar el archivo ejecutable .jar.

Paso 6: Abre tu explorador de archivos (usualmente en la carpeta "Descargas") y verifica que tengas ambos archivos: el .zip y el .jar.

2. Instalación

Paso 1: Descomprime el archivo ZIP descargado. Para ello, haz clic derecho sobre él y selecciona "Extraer todo...".

Paso 2: Entra a la carpeta que se acaba de descomprimir (llamada javacc-javacc-7.0.13). Dentro de ella, crea una nueva carpeta llamada **target** (preferentemente todo en minúsculas para evitar errores).

Paso 3: Mueve o copia el archivo .jar (el que descargaste desde GitHub) dentro de esta nueva carpeta target.

Paso 4: Haz clic derecho sobre el archivo .jar que acabas de mover, selecciona "Cambiar nombre" y nómbralo exactamente **javacc.jar**.

Nota: Si tu explorador de Windows oculta las extensiones de archivo y no ves el ".jar" al final de los nombres, simplemente nómbralo javacc.

3. Configuración de Variables de Entorno

Paso 1: Ve a la carpeta principal descomprimida (javacc-javacc-7.0.13), entra a la subcarpeta llamada scripts y copia la ruta completa desde la barra de direcciones superior del explorador de archivos.

Paso 2: Abre el buscador de Windows (tecla Windows), escribe “variables de entorno” y selecciona la opción “Editar las variables de entorno del sistema”.

Paso 3: En la ventana que se abre, haz clic en el botón inferior llamado “Variables de entorno...”.

Paso 4: En la sección inferior llamada “Variables del sistema”, busca en la lista la variable llamada Path, selecciónala con un clic y luego presiona el botón “Editar...”.

Paso 5: Haz clic en el botón "Nuevo" y pega la ruta de la carpeta scripts que copiaste en el Paso 1. Luego haz clic en Aceptar en todas las ventanas abiertas para guardar los cambios.

Paso 6: Abre una nueva ventana de la terminal (presiona Win, escribe cmd y dale Enter). Es crucial que sea una ventana nueva para que reconozca los cambios.

Paso 7: Escribe el siguiente comando para verificar que la instalación fue exitosa:

```
javacc --version
```

INSTALACIÓN Y CONFIGURACIÓN DE JDK

1. Descarga

Paso 1: Accede a la página oficial de descargas de Oracle Java:

<https://www.oracle.com/mx/java/technologies/downloads/>

Paso 2: Busca la pestaña “Java SE Development Kit” y elige la versión de soporte a largo plazo (LTS) más reciente. Luego, selecciona el instalador correspondiente a tu sistema operativo.

Para Windows, elige la opción "x64 Installer" (descargará un archivo .exe).

Paso 3: Haz clic en el enlace de descarga, acepta los términos y condiciones de Oracle (si la página lo solicita), y espera a que el archivo se descargue por completo.

2. Instalación

Paso 1: Ve a tu carpeta de descargas, haz clic derecho sobre el archivo instalador del JDK (ej. jdk-21_windows-x64_bin.exe) y selecciona “Ejecutar como administrador”.

Paso 2: En la ventana de bienvenida del instalador, presiona “Next” (Siguiente).

Paso 3: La pantalla mostrará la ruta donde se instalará el JDK (por defecto suele ser C:\Program Files\Java\jdk-xx). Es recomendable dejar esta ruta por defecto. Haz clic en “Next” para comenzar la instalación.

Paso 4: Una vez que la barra de progreso termine y veas el mensaje de instalación exitosa, presiona el botón “Close” para cerrar el instalador.

3. Configuración de Variables de Entorno

Paso 1: Abre el explorador de archivos y navega hasta la carpeta donde se instaló el JDK, y entra específicamente a la carpeta bin.

La ruta exacta que debes copiar de la barra superior debería verse similar a esta:

C:\Program Files\Java\jdk-21\bin

(Asegúrate de que termine en \bin, ya que ahí viven los ejecutables).

Paso 2: Abre el buscador de Windows (presiona la tecla Windows), escribe “variables de entorno” y selecciona la opción “Editar las variables de entorno del sistema”.

Paso 3: En la ventana de Propiedades del sistema, haz clic en el botón inferior que dice “Variables de entorno...”.

Paso 4: En el panel de abajo llamado “Variables del sistema”, busca en la lista la variable llamada Path. Haz clic sobre ella para seleccionarla y luego presiona el botón “Editar...”.

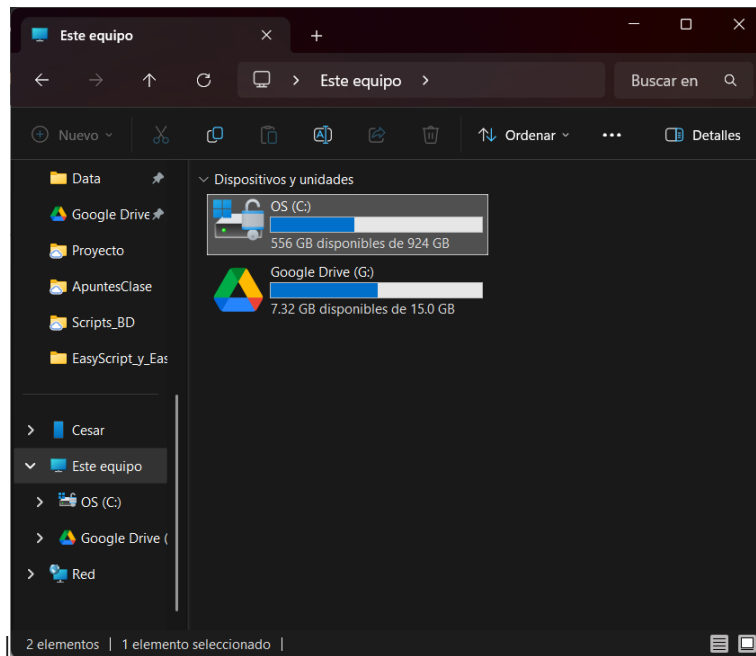
Paso 5: En la nueva ventana, haz clic en el botón "Nuevo". Se habilitará un espacio vacío al final de la lista; pega ahí la ruta de la carpeta bin que copiaste en el Paso 1. Haz clic en "Aceptar" en todas las ventanas que abriste para guardar los cambios.

Paso 6: Abre una nueva ventana de la terminal (escribe cmd en el buscador de Windows y dale Enter). Escribe el siguiente comando para verificar la instalación:

```
java -version
```

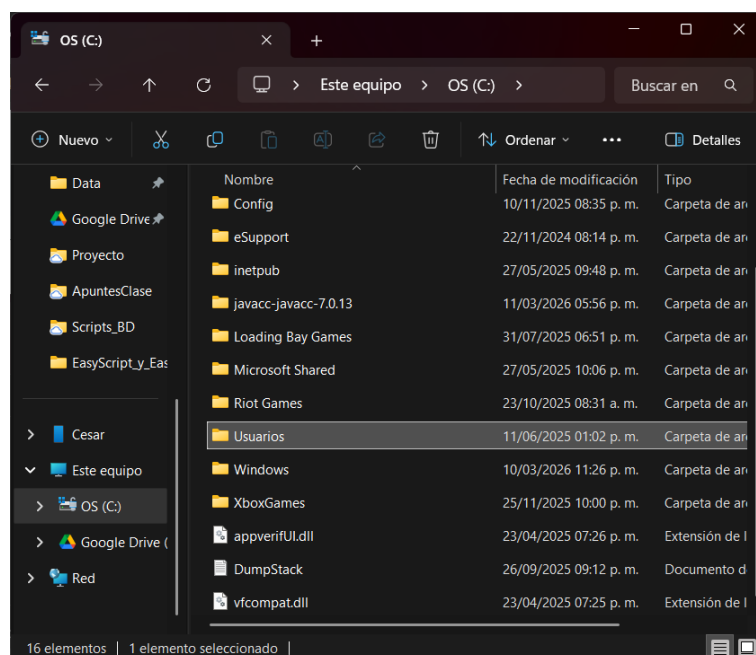
CONFIGURACIÓN DEL ENTORNO DE TRABAJO

1. Abrir el explorador de archivos.
2. Entrar a la carpeta OS(C:) en “Este equipo” y.



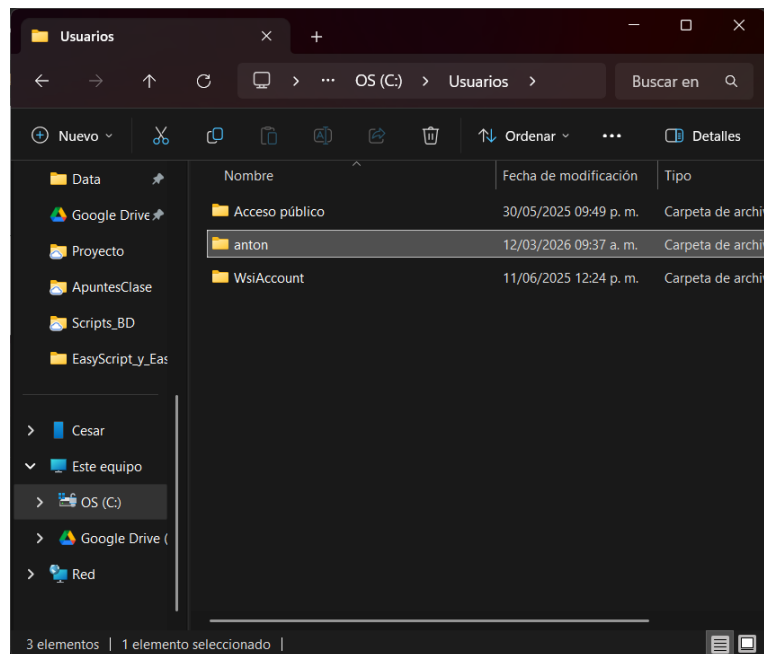
Img 1 Ubicación carpeta OS(C:).

3. Entrar a la carpeta “usuarios”.



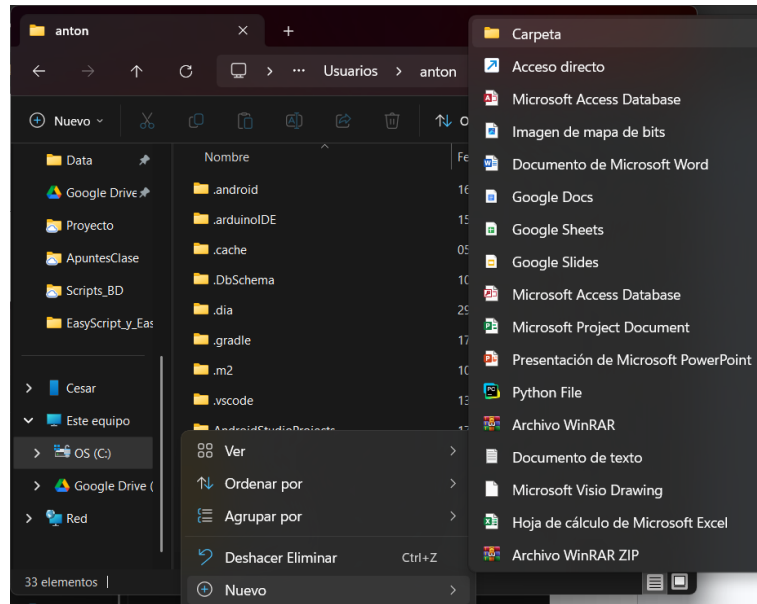
Img 2 Ubicación carpeta “Usuarios”.

4. Identificar la carpeta con el nombre del usuario que está en uso del dispositivo y entrar.



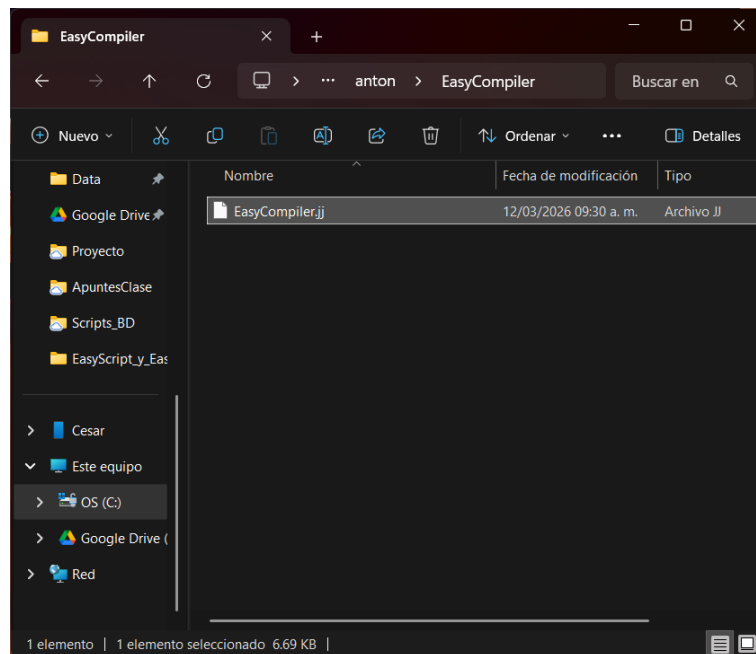
Img 3 Identificar carpeta con nombre de usuario.

5. Crear una carpeta con el nombre del compilador “EasyCompiler”.



Img 4 Creación de la carpeta.

6. Dentro de la carpeta recién creada “EasyCompiler” pegar el archivo “EasyCompiler.jj”.

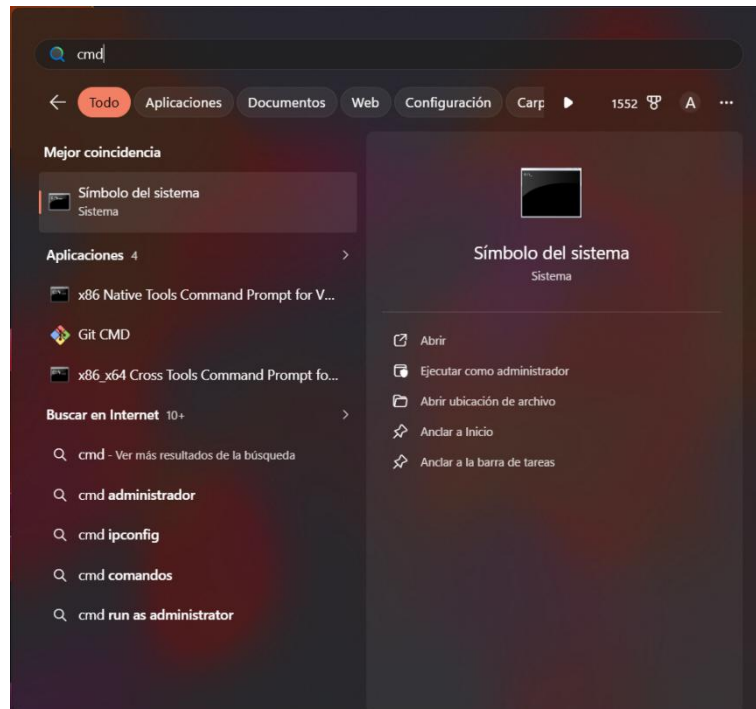


Img 5 Archivo jj en la carpeta “EasyCompiler”.

Nota*: En esta misma carpeta se deben colocar los archivos de prueba.

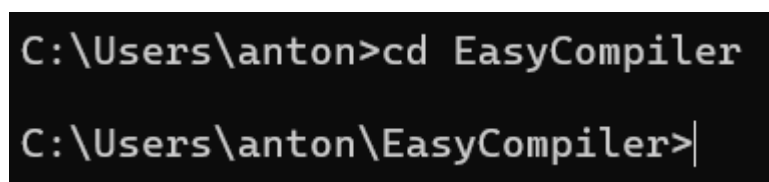
COMPILACIÓN Y EJECUCIÓN PASO A PASO

1. Presiona tecla Windows y buscar CMD para entrar a la consola.



Img 6 Entrar a consola

2. Entrar a la carpeta “EasyCompiler” creada anteriormente.



Img 7 Entrar a carpeta del compilador

3. Activar UTF-8 para ver los acentos correctamente.

```
C:\Users\anton\EasyCompiler>chcp 65001
Página de códigos activa: 65001

C:\Users\anton\EasyCompiler>
```

Img 8 Activar acentos en consola

4. Generar el parser con JavaCC.

```
C:\Users\anton\EasyCompiler>javacc EasyCompiler.jj
Java Compiler Compiler Version 7.0.13 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file EasyCompiler.jj . . .
File "TokenMgrError.java" is being rebuilt.
File "ParseException.java" is being rebuilt.
File "Token.java" is being rebuilt.
File "SimpleCharStream.java" is being rebuilt.
Parser generated successfully.

C:\Users\anton\EasyCompiler>|
```

Img 9 Parser con JavaCC

5. Compilar el código con Java.

```
C:\Users\anton\EasyCompiler>javac *.java

C:\Users\anton\EasyCompiler>
```

Img 10 Compilación del código con Java

6. Ejecutar un archivo de prueba en el compilador.

```
C:\Users\anton\EasyCompiler>Java Main codigo1.txt
```

Img 11 Realizar prueba

SENTENCIAS DEL LENGUAJE

ESTRUCTURA DE PROGRAMA

Cada lenguaje debe iniciar con la palabra clave INICIO y terminar con FIN, habiendo entre ellas las sentencias delimitadas por llaves {}.

SINTAXIS:

```
INICIO {  
  
    // Bloque de código  
  
} FIN
```

COMENTARIOS

Para redactar comentarios de una línea a lo largo del código se debe colocar //, lo que esté después de la doble diagonal será ignorado por el compilador. Para comentar un bloque de líneas estas deberán estar entre /* y */.

SINTAXIS:

```
// Este es un comentario de una línea  
  
/*  
    Este es un  
    bloque de líneas  
    comentadas  
*/
```

TIPOS DE DATOS

El compilador de EasyScript maneja cinco tipos de datos nativos. Para hacer uso de ellos, se deben emplear sus respectivas palabras reservadas al momento de declarar variables o arreglos:

- **ENT:** Representa valores numéricos enteros (positivos y negativos) sin punto decimal.
 - *Ejemplos válidos:* 0, -5, 2026
- **DEC:** Representa valores numéricos con precisión decimal (positivos y negativos).
 - *Ejemplos válidos:* 1.4, 194.45, -45.3
- **LETRA:** Representa un único carácter alfanumérico (letras mayúsculas, minúsculas o números del 0-9). Debe declararse estrictamente entre comillas simples (").
 - *Ejemplos válidos:* 'A', 'z', '5'
- **TXT:** Representa una cadena de texto (conjunto de caracteres). Debe declararse estrictamente entre comillas dobles ("").
 - *Ejemplos válidos:* "Hola mundo", "Edad: 43"
- **BOOL:** Representa un estado lógico. Únicamente admite dos valores predefinidos en el lenguaje.
 - *Valores admitidos:* VERDADERO, FALSO

OPERADORES

Aritméticos:

+ Suma

- Resta

* Multiplicación

/ División

% Módulo

Relacionales:

< Menor que

> Mayor que

<= Menor o igual que

>= Mayor o igual que

== Igual de comparación

!= Diferente de

Lógicos:

&& representa el operador lógico y

|| representa el operador lógico o

!! representa el operador lógico no

DELIMITADORES

{ } bloques de código

() parámetros y agrupaciones

[] arreglos o matrices

, separa valores

: Inicio de un caso para la condicional SEGÚN

; fin de sentencia

DECLARACIÓN DE VARIABLES

Toda variable debe comenzar por una letra mayúscula y continuar sólo con letras minúsculas, números o guiones bajos permite usar acentos del español (á, é, í, ó, ú) y la letra ñ. Pueden declararse en cualquier parte de las sentencias.

SINTAXIS:

```
ENT Variable1 = 10;
```

```
ENT Variable_con_guión = 20;
```

DECLARACIÓN DE CONSTANTES

Toda constante debe comenzar por el prefijo CONST_, seguido del nombre, respetando las mismas reglas de escritura de las variables. Pueden declararse en cualquier parte de las sentencias.

SINTAXIS:

```
ENT CONST_Materias = 6;
```

```
ENT CONST_Materias_con_guion = 20;
```

ASIGNACIÓN:

Brinda la posibilidad de cambiar el valor a una variable o constante a lo largo de las sentencias.

SINTAXIS:

```
Variable1 = Variable1 + 1;
```

INCREMENTO Y DECREMENTO

Forma de sumar o restar 1 a una variable dentro de un ciclo PARA.

SINTAXIS:

```
Incremento: Variable1 ++;
```

```
Decremento: Variable1 --;
```

ARREGLOS Y MATRICES:

Permite declarar matrices bidimensionales de dos maneras, con el número de filas y columnas que tendrá o inicializando directamente la matriz con los valores deseados y arreglos de la misma forma, con el número de elementos o con sus valores inicializados.

SINTAXIS:

```
ARREGLO tipo nombre [tamaño];
```

o

```
ARREGLO tipo nombre = {valores iniciales};
```

```
MATRIZ tipo nombre [filas][columnas];
```

o

```
MATRIZ tipo nombre = {{valores iniciales},{valores  
iniciales}};
```

EJEMPLO:

```
ARREGLO ENT Alumnos [9];                                o

ARREGLO DEC Temperatura = {32.5,40.3,17.2,10.5};

MATRIZ ENT Matriz1 [3][4];                                o

MATRIZ DEC Calificaciones = {{7.5,2.4,8.0},
                               {4.8,5.5,10.0}};
```

IMPRESIÓN A PANTALLA

Imprime información redactada entre comillas dobles “”. Cuando se trata de imprimir variables o constantes deben ir fuera de las comillas y para combinarlo con texto se usa el símbolo + de concatenación. Puedes realizar saltos de con \t (tabulador), \n (salto de línea) o \r, los saltos deben estar establecidos dentro de las comillas.

SINTAXIS:

```
Mensaje: IMPRIMIR (“Este es un mensaje de ejemplo”);

Imprimir variable/constante: IMPRIMIR(Variable1);

Mensaje combinado: IMPRIMIR (“Cantidad de materias: ” +
CONST_Materias);
```

LECTURA DE DATOS

Lee el valor introducido por el usuario y lo asigna a una variable. Se puede agregar texto visible en la terminal al momento de capturar concatenado con la variable. Solo permite el uso de variables, por lo tanto, no reconocerá constantes.

SINTAXIS:

```
LEER (Variable1);

LEER (“Ingrese el dato a evaluar: ” + Variable1);
```

CONDICIONAL SI / CONTRARIO / SINO

Ejecuta bloques de códigos de acuerdo a la condición establecida.

SINTAXIS:

```
SI (condición) {  
    // bloque de código  
} CONTRARIO (otra_condición) {  
    //bloque de código  
} SINO {  
    //bloque de código  
}
```

CONDICIONAL SEGÚN

Evalúa una condición y de acuerdo a dicha condición determinará un caso entre muchos, la variable puede ser de cualquier tipo.

SINTAXIS:

```
SEGUN (Variable1) {  
    CASO valor1:  
        // instrucciones  
        DETENER;  
    CASO valor2:  
        // instrucciones  
        DETENER;  
    DEFECTO:  
        // instrucciones  
}
```


BUCLE PARA

Bucle que repite un bloque de código una cantidad determinada de veces.

SINTAXIS:

```
    PARA (inicialización; condición; incremento) {  
        // bloque de código  
    }
```

BUCLE MIENTRAS

Bucle que repite un bloque de código mientras se cumpla una condición.

SINTAXIS:

```
    MIENTRAS (condición) {  
        // bloque de código  
    }
```

ERRORES LÉXICOS

En EasyScript, un error léxico ocurre cuando el analizador encuentra un carácter o una secuencia de caracteres que no pertenece al alfabeto del lenguaje y no coincide con ninguna de las reglas de escritura definidas.

Cuando esto sucede, el proceso de compilación se detiene al no poder clasificar el símbolo leído.

Causas comunes y soluciones

a) Cadenas de texto no cerradas

- Descripción: Si se abre una comilla doble (") para iniciar un tipo TXT y se llega a un salto de línea o al final del archivo sin la comilla de cierre, el analizador no puede reconocer el texto completo.
- Ejemplo de error: "Hola mundo
- Solución: Asegurarse de cerrar la cadena en la misma línea: "Hola mundo"

b) Caracteres no cerrados

- Descripción: Ocurre al abrir una comilla simple (') para un tipo CAR sin cerrarla adecuadamente.
- Ejemplo de error: 'a
- Solución: Cerrar el carácter con la comilla simple correspondiente: 'a'

c) Literal de carácter inválido

- Descripción: El tipo de dato CAR está definido para aceptar un solo carácter alfanumérico. Si se introducen dos caracteres o símbolos especiales no definidos, el analizador fallará.
- Ejemplo de error: 'abc' o '#'
- Solución: Reducir el contenido a un solo carácter alfanumérico. Ejemplo: 'A'. Si se requiere más de un símbolo, utilizar el tipo TXT con comillas dobles.

d) Identificador mal formado

- Descripción: La sintaxis de EasyScript exige que toda variable comience obligatoriamente con una letra mayúscula. Si se inicia con una minúscula o un guion

bajo (y no existe una palabra reservada que coincida), el analizador lo detectará como un elemento no válido.

- Ejemplo de error: `variable1 = 10;` o `_Variable = 5;`
- Solución: Cambiar la letra inicial por una mayúscula. Ejemplo: `Variable1 = 10;`

e) Operador lógico incompleto o inválido

- Descripción: El compilador utiliza representaciones específicas para sus operadores lógicos (&&, ||, !). Ingresar símbolos aislados que no forman un operador completo generará un error de reconocimiento.
- Ejemplo de error: `SI (! Variable)`
- Solución: Utilizar la sintaxis correcta del lenguaje. Ejemplo: `SI (!! Variable)`

f) Símbolos no reconocidos

- Descripción: Uso de caracteres especiales que no están contemplados en la gramática del lenguaje (por ejemplo: @, #, \$ o ~).
- Ejemplo de error: `#Variable`
- Solución: Eliminar los caracteres extraños y utilizar únicamente la sintaxis y palabras reservadas de EasyScript.

ERRORES SINTÁCTICOS

Un error sintáctico ocurre cuando la secuencia de componentes léxicos (tokens) no respeta las reglas gramaticales de EasyScript. Aunque cada elemento por separado sea válido, su combinación está mal formada. El compilador detecta estas situaciones durante el análisis sintáctico y normalmente muestra un mensaje indicando qué token era esperado y cuál se encontró.

Causas comunes y soluciones

a) Falta de punto y coma al final de una sentencia

- Descripción: Toda sentencia debe terminar con ;. Si se omite, el analizador no sabe dónde acaba la instrucción.
- Ejemplo de error:

```
ENT Valor = 5    // falta ';'

```

```
IMPRIMIR("Hola");

```

- Solución: Agregar el ; al final de cada sentencia.

b) Llaves desbalanceadas

- Descripción: El número de { no coincide con el de } en un bloque. Puede sobrar una llave de cierre o faltar una de apertura.
- Ejemplo:

```
SI (Valor > 0) {

```

```
    IMPRIMIR("Positivo");

```

```
    // falta '}'

```

- Solución: Verificar que cada { tenga su correspondiente }.

c) Paréntesis no cerrados en condiciones o llamadas

- Descripción: En estructuras como SI (, MIENTRAS (, PARA (debe existir el paréntesis de cierre.
- Ejemplo:

```
SI (Edad > 18 { ... }
```

- Solución: Cerrar correctamente: SI (Edad > 18) { ... }

d) Declaración de variable sin tipo de dato

- Descripción: EasyScript exige que toda declaración inicie con un tipo (ENT, DEC, CAR, etc.).
- Ejemplo:

```
Numero = 10;
```

- Solución: Escribir el tipo correspondiente: ENT Numero = 10;

e) Palabras clave mal ubicadas

- Descripción: Usar CONTRARIO sin un SI previo, o escribir SINO sin que exista un SI o CONTRARIO.
- Ejemplo:

```
CONTRARIO (Valor == 0) {  
  
    IMPRIMIR("Cero");  
  
}
```

- Solución: La estructura debe ser SI ... CONTRARIO ... SINO de forma anidada correctamente.

f) Faltar la palabra DETENER en un CASO del SEGUN

- Descripción: Cada CASO debe terminar con DETENER; para indicar la salida del bloque SEGUN.

- Ejemplo:

```
SEGUN (Op) {

CASO 1:

IMPRIMIR("Uno");

// falta DETENER;

DEFECTO:

IMPRIMIR("Otro");

}
```

- Solución: Añadir DETENER; al final de cada caso.

g) Incremento o decremento fuera de un PARA

- Descripción: Si el compilador no admite `Variable++` como sentencia independiente, aparecerá un error sintáctico al encontrarlo fuera de la cabecera de un PARA.
- Ejemplo:

```
INICIO {

ENT Contador = 0;

Contador++; // error '++' no es sentencia independiente

IMPRIMIR ("Contador: " + Contador);

} FIN
```

- Solución: Usar una asignación equivalente (`Contador = Contador + 1;`).

h) Expresiones aritméticas incompletas

- Descripción: Una expresión como Resultado = 5 + o Valor = * 3 genera un error porque falta un operando o un operador está mal colocado.

- Ejemplo:

```
ENT Resultado = 5 + ; // falta el operando derecho
```

```
ENT Valor = * 3; // falta el operando izquierdo
```

```
ENT X = 10 / ; // división sin segundo operando
```

- Solución: Revisar que la expresión tenga la forma Valor Operador Valor.

i) Faltar la palabra FIN al final del programa

- Descripción: Todo programa debe cerrarse con } FIN. Si se omite, el analizador indicará que esperaba FIN.

- Ejemplo:

```
INICIO {
```

```
    IMPRIMIR("Hola mundo");
```

```
} // falta la palabra FIN
```

- Solución: Asegurarse de escribir } FIN al terminar el bloque INICIO.

ERRORES DE INSTALACIÓN Y CONFIGURACIÓN

Esta sección describe los problemas más comunes que pueden presentarse antes de poder utilizar EasyCompiler, específicamente durante los procesos de instalación de las herramientas requeridas y la configuración del entorno del sistema operativo.

a) El comando "javacc" o "java" no se reconoce

- **Descripción:** Al intentar ejecutar `javacc --version` o `java -version` en la consola (CMD), el sistema devuelve un mensaje indicando que no se reconoce como un comando interno o externo, programa o archivo por lotes ejecutable.
- **Causa:** Las rutas de la carpeta `scripts` de JavaCC y/o la carpeta `bin` del JDK no se agregaron correctamente a la variable de entorno **Path** del sistema; o bien, la consola se abrió antes de guardar los cambios en las variables de entorno.
- **Solución:**
 1. Verifique en las "Variables de entorno" que la ruta introducida en el **Path** sea exactamente donde se encuentran los archivos ejecutables (asegúrese de incluir la carpeta `scripts` para JavaCC y la carpeta `bin` para el JDK, como se indica en la sección de instalación).
 2. Cierre la ventana actual de la consola de comandos (CMD) y abra una nueva. Las consolas abiertas previamente no reconocen los cambios recientes en las variables de entorno.

b) Error "Could not find or load main class" al ejecutar JavaCC

- **Descripción:** Al ejecutar `javacc EasyCompiler.jj`, se muestra un error indicando que Java no puede encontrar la clase principal.
- **Causa:** El archivo `.jar` de JavaCC (descargado desde GitHub) no se renombró correctamente a `javacc.jar` o no se colocó dentro de la carpeta `target` creada en el directorio de instalación de JavaCC.
- **Solución:**
 1. Diríjase a la carpeta donde descomprimió JavaCC (ej. `javacc-javacc-7.0.13`).
 2. Verifique que exista una carpeta llamada exactamente `target` (en minúsculas).
 3. Asegúrese de que el archivo ejecutable de JavaCC esté dentro de esa carpeta y se llame exactamente `javacc.jar`.

c) No se genera la carpeta "target" automáticamente

- **Descripción:** El usuario busca la carpeta target dentro del directorio de JavaCC pero no existe.
- **Causa:** En las versiones recientes descargadas del repositorio de código fuente, esta carpeta no viene precreada.
- **Solución:** Siga el **Paso 2** de la sección de Instalación de JavaCC de este manual: debe crear manualmente la carpeta con el nombre target dentro del directorio principal de JavaCC.

ERRORES DE EJECUCIÓN

Los errores de ejecución se presentan cuando EasyCompiler ya está instalado y funcionando, pero ocurre un problema durante el proceso de compilación de un archivo .jj o al intentar evaluar un archivo de código EasyScript (.txt).

a) El archivo ".jj" no se encuentra

- **Descripción:** Al intentar ejecutar javacc EasyCompiler.jj, la consola muestra un error de tipo "File not found".
- **Causa:** La consola (CMD) no está ubicada en el directorio correcto donde se guardó el archivo EasyCompiler.jj, o el archivo tiene un nombre diferente.
- **Solución:**
 1. Asegúrese de haber navegado con el comando cd (Change Directory) hasta la carpeta exacta donde guardó el archivo (ej. C:\Users\SuUsuario\EasyCompiler).
 2. Compruebe que el nombre del archivo sea exactamente EasyCompiler.jj y no tenga extensiones ocultas como EasyCompiler.jj.txt.

**b) Error al compilar con "javac .java"*

- **Descripción:** La consola arroja múltiples errores de código al ejecutar javac *.java.
- **Causa:** El archivo .jj tiene errores lógicos en su estructura interna definidos por el programador del compilador, o no se generaron correctamente los archivos .java en el paso anterior con JavaCC.
- **Solución:** Vuelva a ejecutar javacc EasyCompiler.jj. Si este paso muestra errores (como Tokens mal definidos en el archivo fuente), estos deben ser corregidos en el código fuente de EasyCompiler.jj antes de intentar compilar con Java.

c) Caracteres extraños en los mensajes de la consola (Problemas de codificación)

- **Descripción:** Los acentos o caracteres especiales (como la 'ñ') en los mensajes de error de EasyCompiler se muestran como símbolos extraños en la consola de Windows.
- **Causa:** La página de códigos activa de la consola no soporta la codificación UTF-8.
- **Solución:** Antes de ejecutar el compilador, escriba en la consola el comando chcp 65001 y presione Enter (como se indica en el Paso 3 de Compilación y Ejecución). Esto activará la visualización correcta de caracteres UTF-8.

PREVENCIÓN DE ERRORES

Para garantizar una experiencia fluida y evitar la mayoría de los problemas documentados anteriormente, se recomienda a los usuarios seguir estas buenas prácticas al trabajar con EasyCompiler:

- **Siga el orden estricto de instalación:** No intente ejecutar el compilador sin antes haber verificado exitosamente la instalación de Java (JDK) y JavaCC mediante los comandos de versión (java -version y javacc --version).
- **Organización de archivos:** Mantenga siempre el archivo EasyCompiler.jj y todos sus archivos de prueba (ej. codigo1.txt) dentro de la misma carpeta dedicada (ej. C:\Users\Anton\EasyCompiler). Esto evita confusiones con las rutas en la consola.
- **Nombres descriptivos y sin espacios:** Al crear archivos de prueba para compilar, utilice nombres sin espacios y evite caracteres especiales (ej. use errores_comunes.txt en lugar de errores_comunes!.txt).
- **Atención a las extensiones:** Tenga cuidado si Windows oculta las extensiones de los archivos por defecto. Es posible que un archivo de texto creado como código fuente se llame realmente programa.txt.txt sin que usted lo note, lo que causará errores al intentar leerlo desde la consola.
- **Verificación visual del código:** Antes de ejecutar un archivo de prueba en EasyCompiler, revise visualmente que todas las llaves {}, paréntesis () y comillas "" o " estén correctamente abiertas y cerradas. El 80% de los errores de compilación reportados se deben a delimitadores sin cerrar.
- **Reinicio de consola:** Si modifica alguna variable de entorno del sistema operativo, siempre cierre la consola (CMD) actual y abra una nueva; de lo contrario, los cambios no tendrán efecto.

REFERENCIAS BIBLIOGRÁFICAS

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2a ed.). Pearson Educación.
2. Appel, A. W. (2002). *Modern compiler implementation in Java* (2a ed.). Cambridge University Press.
3. Brookshear, J. G. (1993). *Teoría de la computación: Lenguajes formales, autómatas y complejidad*. Addison-Wesley Iberoamericana.
4. Cooper, K. D., & Torczon, L. (2011). *Engineering a compiler* (2a ed.). Morgan Kaufmann.
5. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3a ed.). Pearson Educación.
6. Martin, J. C. (2004). *Lenguajes formales y teoría de la computación* (3a ed.). McGraw-Hill.